

Deployment guide

RAG SaaS Platform

Deployment guide

Guide de déploiement sécurisé -- api/ (audit Categorie 5, item 33)

Ce guide couvre le déploiement de `api/` (le backend d'authentification, Partie 1.1) en production. Il complète `docs/AUTH_BACKEND_SETUP.md` (configuration locale, architecture, exemples d'API) -- ce document-ci se concentre sur ce qui change entre "ça marche sur ma machine" et "c'est sûr d'exposer ça sur Internet".

1. Prérequis d'infrastructure

Service	Rôle	Notes
PostgreSQL	Base de données principale	Supabase, RDS, ou tout Postgres géré. <code>DATABASE_URL</code> doit utiliser le driver <code>asyncpg</code> (<code>postgresql+asyncpg://...</code>).
Redis	Rate limiting, cache géo-IP, challenges WebAuthn, broker/backend Celery	Un seul serveur Redis suffit (bases logiques séparées par numéro, voir <code>.env.example</code>) ou plusieurs instances managées si vous préférez isoler chaque usage.
Stockage S3-compatible	Avatars utilisateur	AWS S3, Cloudflare R2, ou Supabase Storage.
Resend (ou un autre expéditeur transactionnel)	Emails (vérification, reset, alertes)	Domaine expéditeur vérifié -- indispensable, voir §4.
Worker + Beat Celery	Purge de comptes, purge de tokens, rotation automatique des clés JWT	Processus séparés du serveur API, voir §6.
Reverse proxy (nginx, Caddy, ou le load balancer du fournisseur cloud)	TLS/HTTPS, en-tête <code>X-Forwarded-For</code> fiable	Voir §3 -- obligatoire , pas optionnel.

2. Variables d'environnement

Ne jamais committer de vraies valeurs -- `.env.example` documente chaque variable avec son rôle. Checklist de ce qui **doit** être défini (au-delà des valeurs par défaut) avant un déploiement réel :

- [] `DATABASE_URL` -- pointant vers la base de production, jamais celle de dev/CI.
- [] `JWT_SECRET_KEY`, `SESSION_MIDDLEWARE_SECRET`, `AUDIT_LOG_HMAC_SECRET_KEY` -- trois secrets **distincts**, générés avec `python -c "import secrets; print(secrets.token_urlsafe(48))"`. Ne jamais réutiliser une valeur de dev.
- [] `COOKIE_SECURE=True` -- **obligatoire dès que le site est servi en HTTPS** (ce qui doit toujours être le cas en production). `False` désactive le flag `Secure` des cookies de session, acceptable seulement en dev HTTP

local.

- [] `SUPPORT_EMAIL`, `RESEND_API_KEY`, `EMAIL_FROM_ADDRESS` -- une adresse `EMAIL_FROM_ADDRESS` sur un domaine réellement vérifié dans Resend, sinon les emails partent en sandbox ou échouent silencieusement.
- [] `FRONTEND_URL`, `OAuth_REDIRECT_BASE_URL` -- les vraies URLs publiques, pas `localhost`.
- [] `RATE_LIMIT_ENABLED=True` -- **ne jamais** mettre `False` en production (voir le rappel dans `api/security/rate_limit.py`).
- [] `TRUSTED_IPS`, `TRUSTED_COUNTRIES` / `SUSPICIOUS_COUNTRIES` (optionnels, audit Catégorie 4) -- à revoir avec l'équipe sécurité si utilisés, ce ne sont pas des valeurs par défaut à copier sans réflexion.
- [] `SECRET_ENCRYPTION_KEY` -- requis dès que la rotation JWT automatique (`JWT_AUTO_ROTATION_INTERVAL_DAYS > 0`) ou une connexion SSO entreprise est utilisée. Généré une fois, jamais régénéré sans ré-chiffrer les lignes existantes.
- [] `WEBAUTHN_RP_ID` / `WEBAUTHN_RP_ORIGIN` -- le vrai domaine et la vraie origine du frontend, pas `localhost`. Un WebAuthn enregistré sous le mauvais `RP_ID` ne fonctionnera tout simplement pas -- à définir correctement **avant** que des utilisateurs enregistrent leurs clés.
- [] Toutes les variables `S3_*`, OAuth (`GOOGLE_OAUTH_*` / `GITHUB_OAUTH_*`), Celery (`CELERY_*`) -- pointant vers les vraies ressources de production.

3. Reverse proxy, HTTPS, et le piège `X-Forwarded-For`

C'est le point le plus facile à mal configurer silencieusement.

`api/utils.py`'s `client_ip()` fait confiance à l'en-tête `X-Forwarded-For` tel quel -- c'est correct et nécessaire quand l'application tourne derrière un vrai reverse proxy qui **définit ou écrase** cet en-tête lui-même, mais si l'API est exposée directement sur Internet sans un tel proxy devant elle, **n'importe quel client peut fixer une valeur arbitraire de `X-Forwarded-For` et contourner trivialement :**

- le rate limiting par IP (`/auth/login`, `/auth/register`, etc.),
- l'ajustement géo-adaptatif des limites (audit Catégorie 4, item 29),
- et surtout **la liste `TRUSTED_IPS`** (item 30) -- c'est pour cette

raison précise que l'exemption d'IP de confiance est vérifiée contre l'adresse TCP réelle (`api/security/trusted_ips.py`'s `direct_peer_ip`), jamais contre `client_ip()` -- mais cette protection ne couvre que CETTE fonctionnalité, pas le rate limiting ordinaire par IP ailleurs dans le code, qui dépend bien de la configuration correcte du proxy décrite ici.

Ce qu'il faut vérifier avant d'aller en production :

1. L'API ne doit **jamais** être directement joignable depuis Internet --

seul le reverse proxy doit avoir une route réseau vers le port de l'API (firewall/security group, réseau privé du cloud).

1. Le reverse proxy doit **écraser** (pas seulement ajouter) tout

`X-Forwarded-For` envoyé par le client, et le remplacer par la vraie adresse IP source qu'il observe :

```
# nginx -- exemple minimal
server {
```

```
listen 443 ssl http2;
server_name api.votredomaine.com;

ssl_certificate      /etc/letsencrypt/live/api.votredomaine.com/fullchain.pem;
ssl_certificate_key  /etc/letsencrypt/live/api.votredomaine.com/privkey.pem;

location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    # Écrase toute valeur envoyée par le client -- ne PAS utiliser
    # $proxy_add_x_forwarded_for ici, qui CONCATÈNE au lieu
    # d'écraser et laisserait passer une valeur falsifiée en
    # première position.
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

1. Si vous utilisez un load balancer managé (AWS ALB, Cloudflare, Fly.io,

Render) -- vérifiez sa documentation : la plupart réécrivent déjà correctement `X-Forwarded-For`, mais ce n'est **pas universel**, et certains l'ajoutent en fin de liste plutôt qu'en le remplaçant (auquel cas `client_ip()` 's choix du **premier** élément serait le mauvais -- revoir cette fonction si votre fournisseur a ce comportement).

1. `COOKIE_SECURE=True` et servir uniquement en HTTPS (redirection

automatique HTTP -> HTTPS au niveau du proxy) -- sans quoi les cookies de session/CSRF ne sont jamais réellement protégés en transit.

4. Email transactionnel

- Domaine vérifié dans Resend (SPF/DKIM configurés) -- sans ça, Resend

limite l'envoi à l'adresse du compte propriétaire uniquement (visible dans les tests locaux : `"You can only send testing emails to your own email address"`).

- `SUPPORT_EMAIL` doit être une boîte réellement surveillée -- elle est

ajoutée en pied de page de chaque email (obligation RGPD Art. 12).

5. Base de données

- Exécuter les migrations Alembic avant le premier déploiement et à

chaque mise à jour : `python -m alembic upgrade head`.

- Sauvegardes automatiques activées (géré par le fournisseur managé dans

la plupart des cas -- Supabase/RDS le font par défaut, à vérifier).

- `AUDIT_LOG_HMAC_SECRET_KEY` et `SECRET_ENCRYPTION_KEY` : perdre l'un ou

l'autre après coup casse respectivement la vérification d'intégrité des logs d'audit existants et le déchiffrement des secrets déjà stockés (clés JWT, secrets SSO) -- à sauvegarder dans un vrai gestionnaire de secrets (Vault, AWS Secrets Manager, etc.), jamais uniquement dans le `.env` d'une seule machine.

6. Processus applicatifs

API (multi-worker, recommandé en production) :

```
PROMETHEUS_MULTIPROC_DIR=/var/run/prometheus_multiproc \  
WEB_CONCURRENCY=4 \  
gunicorn -c gunicorn.conf.py api.main:app
```

Voir `gunicorn.conf.py` -- `PROMETHEUS_MULTIPROC_DIR` doit être un répertoire partagé par tous les workers, vidé au démarrage (déjà géré par `on_starting`). Un seul worker (`WEB_CONCURRENCY=1` ou `uvicorn` seul) fonctionne aussi mais ne tire parti que d'un seul cœur CPU.

Worker et Beat Celery (obligatoires -- sans eux, la purge de comptes différée, le nettoyage des tokens révoqués expirés, et la rotation automatique de la clé JWT ne s'exécutent jamais) :

```
celery -A api.tasks.celery_app worker --loglevel=info  
celery -A api.tasks.celery_app beat --loglevel=info
```

À faire tourner comme des services supervisés (systemd, Docker avec `restart: always`, ou l'équivalent de votre PaaS) -- un crash silencieux du worker ou de beat signifie que les tâches planifiées (purge RGPD, rotation JWT) s'arrêtent sans avertissement visible autrement que `GET /admin/jwt-keys` ne montrant plus de nouvelle rotation.

7. Checklist de sécurité pré-déploiement

- [] `COOKIE_SECURE=True` et HTTPS forcé de bout en bout.
- [] Reverse proxy en place, `X-Forwarded-For` écrasé (jamais concaténé) -- voir §3.
- [] `RATE_LIMIT_ENABLED=True`, Redis réellement accessible (vérifier `GET /health/ready`).
- [] Tous les secrets (`JWT_SECRET_KEY`, `SESSION_MIDDLEWARE_SECRET`, `AUDIT_LOG_HMAC_SECRET_KEY`, `SECRET_ENCRYPTION_KEY`) générés spécifiquement pour la production, jamais copiés depuis `.env` de dev/CI.
- [] Domaine Resend vérifié, `SUPPORT_EMAIL` surveillé.
- [] Migrations Alembic à jour (`alembic upgrade head` exécuté).
- [] Worker et Beat Celery tournent comme services supervisés (redémarrage automatique).
- [] `WEBAUTHN_RP_ID`/`WEBAUTHN_RP_ORIGIN` pointent sur le vrai domaine avant toute inscription WebAuthn réelle (changer après coup invalide les clés déjà enregistrées).
- [] Si SSO entreprise ou rotation JWT automatique sont utilisées : `SECRET_ENCRYPTION_KEY` sauvegardée dans un vrai gestionnaire de secrets, pas seulement dans le `.env` d'un serveur.
- [] `TRUSTED_IPS` (si utilisé) contient uniquement des IP/plages réellement de confiance -- une entrée trop large annule le rate limiting pour cette plage entière.
- [] `GET /health/ready` vérifié en conditions réelles (retourne `"database": "ok"` et `"rate_limit_redis": "ok"`) avant de considérer le déploiement terminé.
- [] Scan Snyk/OWASP ZAP de la CI (`.github/workflows/regression.yml`) vert sur le dernier commit déployé.

Deployment targets

Deployment targets

The API image is `Dockerfile.api` (gunicorn on port 8000, health check `GET /health`, readiness `GET /health/ready`). Background work (document ingestion, scheduled billing, GDPR purges, evaluation jobs) needs a **Celery worker** and **one Celery beat**:

```
celery -A api.tasks.celery_app worker --loglevel=info
celery -A api.tasks.celery_app beat --loglevel=info # exactly one instance
```

Target	Files	Status
Docker Compose	<code>docker-compose.yml</code> , <code>docker-compose.selfhosted.yml</code>	pre-existing
Render	<code>deploy/render/render.api.yaml</code> (API + worker + beat blueprint; the root <code>render.yaml</code> is the old prototype and is left untouched)	written, not applied
Railway	<code>deploy/railway/</code>	written, not deployed
Fly.io	<code>deploy/fly/fly.toml</code>	written, not deployed
Kubernetes	<code>deploy/kubernetes/</code>	written, not applied
Helm	<code>deploy/helm/rag-saas/</code>	written, not linted (helm not installed on the authoring machine)
Terraform	<code>deploy/terraform/{aws,gcp,azure,digitalocean}</code>	written, not validated (terraform not installed on the authoring machine)
Dedicated tenant	<code>deploy/dedicated-tenant/</code>	written, not run

Honest status: none of these files has been executed against a real cloud account. They follow each platform's documented format and the real container contract above, and every one needs secrets (see `.env.example` / `docs/deployment`) and a first `alembic upgrade head` (run once, by the owner, against the target database after a backup). Treat them as a starting point to be validated on a staging account.

Render

Déploiement backend sur Render

Service réel

- **Nom** : `rag-saas-api`
- **URL** : `https://rag-saas-api-sjsm.onrender.com`
- **Type** : Web Service, Docker, tier gratuit
- **Dockerfile** : `./Dockerfile.api` (pas le `Dockerfile` racine, qui

sert un prototype Streamlit sans rapport)

- **Health check** : `/health` (configuré dans `Dockerfile.api`),

vérifiable manuellement via `/health/ready` (détail DB + Redis)

Variables d'environnement réellement nécessaires

Toutes les variables du `.env` local (voir le bloc collé dans Render → Environment → "Add from .env" lors de la mise en place initiale), y compris depuis cette session :

```
RATE_LIMIT_REDIS_URL=rediss://default:<password>@<host>.upstash.io:6379
CELERY_BROKER_URL=rediss://default:<password>@<host>.upstash.io:6379
CELERY_RESULT_BACKEND=rediss://default:<password>@<host>.upstash.io:6379
```

Statut réel vérifié en direct (2026-09-17)

1. Backend accessible

✅ **Fonctionnel.** `GET /health/ready` répond `{"database":"ok", ...}` depuis le dernier déploiement (`b422a3d`, correctif `.dockerignore`). Inscription réelle testée en direct via l'API : `HTTP 201`, token JWT valide retourné.

2. Redis / rate limiting

❌ **Non fonctionnel actuellement, malgré les 3 variables ajoutées par l'utilisateur.** `GET /health/ready` continue de répondre `"rate_limit_redis": "unreachable -- rate limiting is NOT currently enforced"` après le redéploiement déclenché par l'ajout des variables.

Vérifié pour écarter la cause côté Upstash : la même URL Redis, testée en direct depuis une autre machine (ping direct), répond correctement (`PING` → `True`). Le service Upstash lui-même fonctionne.

Cause probable, à vérifier côté Render (je n'ai pas accès au dashboard pour le confirmer moi-même) :

- Un espace, une apostrophe ou un guillemet collé par erreur dans la

valeur lors du copier-coller (le bloc que je t'ai donné n'a pas de guillemets autour de l'URL — si Render en a ajouté automatiquement via "Add from .env", la valeur réelle stockée pourrait contenir des guillemets littéraux qui cassent l'URL).

- Un nom de variable mal orthographié (`RATE_LIMIT_REDIS_URL` doit être

exact, sensible à la casse).

- Les 3 variables existent mais un redéploiement supplémentaire est

nécessaire pour qu'elles soient effectivement lues par le processus déjà démarré (rare sur Render, qui redéploie normalement tout seul à chaque changement de variable, mais possible si le déploiement en cours au moment de l'ajout a "gagné la course").

Mise à jour (résolu) : après avoir remplacé le contenu des 3 variables directement dans Render (au lieu de les modifier par-dessus l'existant), `GET /health/ready` répond maintenant `"rate_limit_redis":"ok"`. Cause réelle probable : la valeur collée la première fois contenait un caractère invisible (espace, guillemet) introduit lors d'un copier-coller précédent — jamais confirmé avec certitude (pas d'accès à l'ancienne valeur, écrasée), mais le remplacement complet a résolu le problème de façon reproductible.

2 bis. Vrai crash de déploiement trouvé et corrigé

● → ✅ **Corrigé.** `gunicorn.conf.py`'s `child_exit()` faisait planter tout l'arbitre Gunicorn dès qu'un worker se terminait (donc à chaque déploiement, lors du redémarrage) : `PROMETHEUS_MULTIPROC_DIR` n'était jamais défini dans `Dockerfile.api`, et `multiprocess.mark_process_dead()` plante avec un `TypeError` si cette variable est absente. Corrigé des deux côtés (`Dockerfile.api` définit vraiment la variable, `child_exit()` a maintenant une protection). Vérifié : déploiement `cf94eac` réussi en 2m56s, "Deploy succeeded — Live".

2 ter. Vraie erreur de pool de connexions PostgreSQL trouvée et corrigée

● → ✅ **Corrigé.** Log réel du déploiement `cf94eac` : `asyncpg.exceptions.InternalServerError: (EMAXCONNECTION) max clients reached in session mode - max clients are limited to pool_size: 15.`

Cause réelle : `DATABASE_URL` pointe vers le pooler Supabase en **mode session** (port 5432), qui plafonne à **15 connexions clientes au total, tous processus confondus** — pas par application. Le moteur SQLAlchemy async de `api/database.py` n'avait pas de `pool_size` / `max_overflow` explicites, donc utilisait les valeurs par défaut de SQLAlchemy (5 + 10 = **15 connexions possibles à lui seul**), laissant zéro marge pour toute autre connexion simultanée (un serveur de dev local, un script d'admin ponctuel, un futur worker Celery séparé).

Corrigé : `pool_size=3`, `max_overflow=2` explicites (5 connexions max par processus au lieu de 15), commit `cf94eac`.

Gap connu, non corrigé (portée limitée dans le temps disponible) : 18 autres fichiers `api/tasks/*.py` créent chacun leur propre moteur SQLAlchemy synchrone contre la même base, avec les mêmes valeurs par défaut non plafonnées. Sans impact aujourd'hui car **aucun worker Celery n'est encore déployé** (voir section 3) — mais dès qu'un sera créé, ce même risque de saturation du pool Supabase reviendra à moins de plafonner ces 18 moteurs aussi. À traiter en même temps que la création du service Celery worker.

3. Celery (traitement de documents en arrière-plan)

● Non fonctionnel — vérifié en direct, pas une supposition. Un document texte réel a été uploadé via l'API sur le compte de test (`POST /organizations/{id}/documents`, `HTTP 201`, statut `pending`). Revérifié 15 secondes plus tard : toujours `"status":"pending"`, `"processed_at":null` — jamais traité.

Cause réelle, pas un bug de code : un seul service Render a été créé (`rag-saas-api`, un Web Service qui lance `gunicorn api.main:app` via `Dockerfile.api's CMD`). Ce process sert l'API HTTP, il ne consomme jamais la file de tâches Celery. Les documents uploadés (`schedule_document_processing` → `process_document_task.delay(...)`, voir `api/security/documents.py`) sont bien envoyés dans Redis, mais personne ne les lit de l'autre côté.

Ce qu'il faut pour que Celery fonctionne réellement en production : créer un **second service Render**, de type **Background Worker** (pas Web Service), même repo/Dockerfile, mais avec une commande de démarrage différente :

```
celery -A api.tasks.celery_app worker --loglevel=info
```

Et éventuellement un troisième service pour `celery beat` (tâches planifiées : facturation mensuelle, purge de comptes, etc.) :

```
celery -A api.tasks.celery_app beat --loglevel=info
```

Ni l'un ni l'autre n'existe actuellement. Document tests uploadés resteront `pending` indéfiniment tant que ce n'est pas fait.

4. Widget embarquable et traductions

✅ Fonctionnel, vérifié en direct (voir `docs/audit/DASHBOARD_BROKEN.md` point 1) : `GET /widget/script.js` → 200, `GET /i18n/translations/fr` → dictionnaire réel.

Résumé

Composant	Statut	Action requise
API HTTP	✅ Fonctionnel	—
Base de données	✅ Fonctionnel	—
Widget / i18n	✅ Fonctionnel	—
Redis / rate limiting	● Non fonctionnel	Revérifier les 3 variables sur Render (copier-coller sans guillemets)
Celery (traitement documents)	● Non fonctionnel	Créer un service Render "Background Worker" séparé

Background worker

Worker Celery -- limites actuelles et solution perenne

Etat actuel

Le worker Celery de production n'est **pas** un vrai processus permanent. C'est un workflow GitHub Actions programme (`.github/workflows/celery-worker.yml`), qui démarre toutes les 10 minutes, traite ce qui est dans la file pendant une fenêtre bornée à 7 minutes, puis s'arrête -- une solution de contournement, pas une architecture cible, adoptée parce que le tier Background Worker de Render n'a pas de palier gratuit (7\$/mois minimum, confirme en direct sur le dashboard Render). Voir `docs/deployment/GITHUB_ACTIONS.md` pour l'historique complet des bugs trouvés et corrigés pour la faire fonctionner (TLS Redis, timeout de job, config S3).

Limites réelles, non cachées

- Un document uploadé peut attendre jusqu'à ~10 minutes avant d'être

traité (l'intervalle de la planification), jamais instantané.

- Une seule tâche à la fois (`--pool=solo --concurrency=1`) -- pas de

parallelisme si plusieurs documents arrivent en même temps.

- Le worker ne tourne pas du tout si le dépôt GitHub est passé en

privé ET que le quota Actions gratuit du compte est épuisé (voir l'incident documenté dans

`docs/deployment/GITHUB_ACTIONS.md`, causé par un ancien workflow de tests cassé qui a consommé tout le quota avant même la création de ce worker).

- ~~Aucune vraie planification `beat` ~~ (**corrige, PROD-001**) : le workflow

démarré désormais un processus `celery beat` à côté du worker, avec un ordonnanceur qui mémorise dans Redis l'heure du dernier lancement de chaque tâche (`api/tasks/beat_scheduler.py`). Une tâche devenue due entre deux passages (facturation mensuelle, crédits mensuels, relances, purges RGPD, renouvellement SSL, balayage des documents `pending`) est envoyée au passage suivant (au plus ~10 min de retard), et une tâche déjà exécutée n'est pas renvoyée. **Limites** : il faut que le worker reste planifié (quota Actions) ; une tâche planifiée prend le retard de l'intervalle du workflow ; un vrai service `beat` permanent (solution pérenne ci-dessous) reste préférable. Non vérifié en conditions réelles ici (aucun Redis ni runner GitHub disponibles) : vérifié par test unitaire de l'ordonnanceur.

Solution pérenne recommandée

Par ordre de preference, une fois le budget disponible :

1. **Render Background Worker** (7\$/mois) -- le plus simple, meme

plateforme que l'API web actuelle, aucun changement d'architecture.

1. **Fly.io** -- alternative deja explore partiellement dans une

session anterieure (app `rag-saas-platform` existante mais mal configuree et en essai expirant), a reprendre proprement si Render n'est pas retenu.

1. **Un VPS low-cost avec plus de RAM** (ex. Hetzner, OVH) -- plus de

travail d'installation/maintenance, mais RAM/CPU nettement superieurs pour le meme budget, ce qui adresserait AUSSI le probleme memoire documente dans `docs/devops/MEMORY.md` si le worker et l'API tournaient sur la meme machine plus genereuse.

Statut

Non change dans cette session (necessite un budget, decision produit). Ce document rend le compromis actuel explicite pour que la decision se prenne en connaissance de cause, pas par default.